

PORTFOLIO

박인호 | Game Programmer

간단한 코딩 스타일을 알 수 있는 미션 시스템 예제

UE 5.5.4 Environment | Mediator & Observer Pattern Applied

MISSION SYSTEM DESIGN GOALS

- </> **UI와 도메인 로직의 완전한 분리**
핵심 연산과 연출의 독립으로 유지보수 비용 절감
- </> **객체 간 직접 참조 최소화**
의존성을 제거하여 예기치 못한 사이드 이펙트 방지
- </> **상태 기반(State-driven) 설계**
미션 라이프사이클의 명확한 관리와 흐름 제어
- </> **확장성 및 유지보수성 확보**
새로운 미션 타입 추가가 용이한 유연한 구조

"Simple, Decoupled, Stable"

가독성 높은 코드는 비즈니스 로직의 명확성에서 시작됩니다.

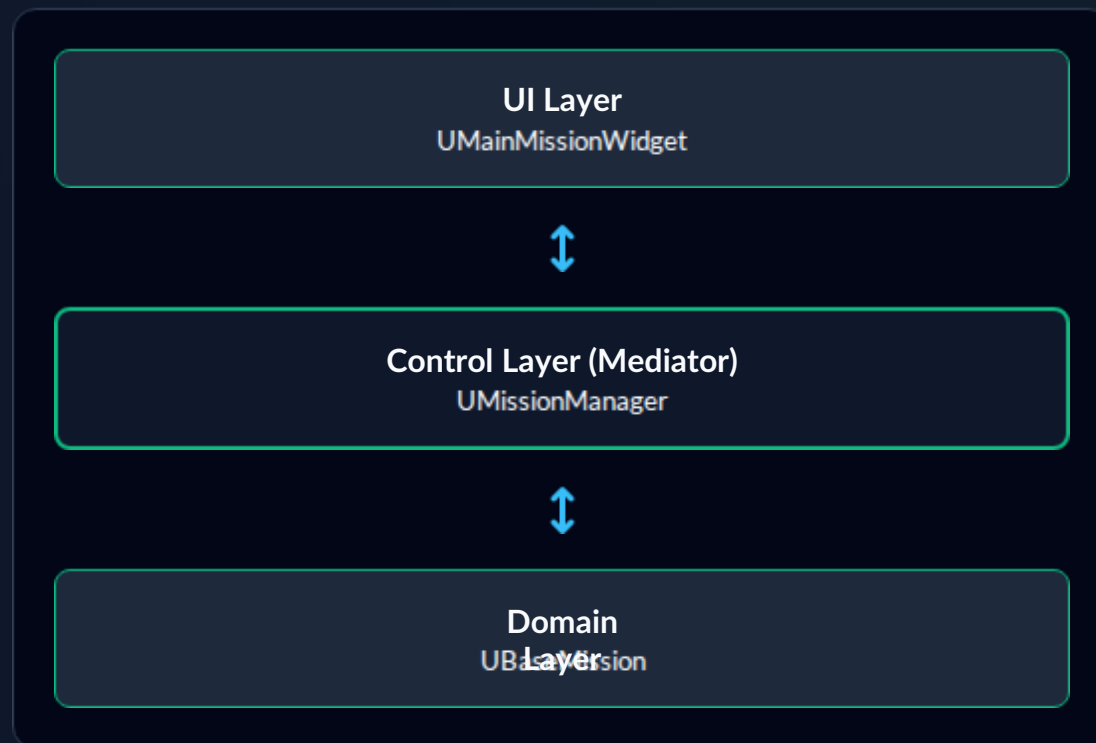
ARCHITECTURE & COMMUNICATION FLOW

3-Layer + Mediator Structure

Widget과 Mission은 서로를 전혀 모르는 상태로 유지되며, **MissionManager**가 모든 통신을 중재합니다.

</> 명령 흐름 (Command): 상위(Widget) → 중재자 → 하위 (BaseMission)로의 함수 호출

</> 이벤트 흐름 (Event): 하위(BaseMission) 상태 변경 → Delegate Broadcast → 상위(Widget) 갱신



CORE DESIGN PATTERNS



Mediator Pattern

모든 객체 통신을 매니저가 집중 관리합니다. 직접 참조를 제거하여 객체 교체 및 로직 수정 시 영향 범위를 최소화합니다.



Observer Pattern

Delegate를 사용하여 상태 변경을 전파합니다.
Event-driven 방식으로 CPU 자원을 최적화합니다.

STATE-BASED COMMAND LOGIC

미션 상태 (State)	Left Button Action (Command)	Right Button Action (Command)
Ready (대기)	A AgreeMission()	Disagree()
InProgress (진행 중)	SuccessMission()	FailedMission()

상태에 따라 동일한 입력 장치(Button)가 각기 다른 도메인 로직을 수행하도록 설계되었습니다.

THANK YOU

안정적이고 확장 가능한 코드, 팀의 자산이 되는 개발자가 되겠습니다.

 hello1013@naver.com

 010-6782-1013

 https://github.com/inhopark/MissionSystem_UE5.git